

VIDEO_TUTORIAL_SCRIPTS

HelixCode Video Tutorial Scripts

This document contains scripts for video tutorials covering HelixCode deployment and usage.

Tutorial 1: Getting Started with HelixCode (10 minutes)

Script

[INTRO - 0:00-0:30]

“Welcome to HelixCode! In this tutorial, we’ll get you up and running with HelixCode in just 10 minutes. HelixCode is an enterprise-grade distributed AI development platform that enables intelligent task division, work preservation, and cross-platform development workflows.”

[WHAT YOU’LL LEARN - 0:30-1:00]

“By the end of this video, you’ll know how to: - Install HelixCode on your system - Start the server - Create your first project - Run a simple task - View the results in the web UI”

[PREREQUISITES - 1:00-1:30]

“Before we begin, make sure you have: - Go 1.24 or later installed - PostgreSQL 14 or later - At least 4GB of RAM - Basic command line knowledge”

[INSTALLATION - 1:30-3:00]

“Let’s start by cloning the repository:

```
git clone https://github.com/helixcode/helixcode.git
cd HelixCode
```

Now, let’s install dependencies:

```
go mod download
```

Build the main server:

```
make build
```

This creates the binary in the bin directory. Now let's set up the database. Create a new PostgreSQL database:

```
createdb helixcode
createuser helixcode
```

Set your database password as an environment variable:

```
export HELIX_DATABASE_PASSWORD=your_password
export HELIX_AUTH_JWT_SECRET=your_secret_key
````
```

```
[STARTING THE SERVER - 3:00-4:00]
```

"Now we can start the server:

```
```bash
./bin/helixcode
```

You should see output indicating the server is running on port 8080. Open your browser and navigate to <http://localhost:8080>. You'll see the HelixCode web interface."

[CREATING A PROJECT - 4:00-6:00]

"Let's create your first project. Click 'New Project' in the UI. Give it a name like 'My First Project' and add a description. Click Create.

Now we'll add a task. Click 'New Task' and fill in: - Title: 'Set up development environment' - Type: Planning - Priority: High - Description: 'Configure local development setup'

Click Create Task. Your task is now in the queue!"

[MONITORING PROGRESS - 6:00-7:30]

"Navigate to the Dashboard to see your project status. You can see: - Active tasks - Worker status - System metrics - Recent activity

Click on your task to see detailed information including logs, status, and progress."

[API EXAMPLE - 7:30-9:00]

"You can also interact with HelixCode via API. Let's create a task programmatically:

```
# First, log in
TOKEN=$(curl -X POST http://localhost:8080/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"admin123"}' | jq -r
    '.token')

# Create a task
curl -X POST http://localhost:8080/api/v1/tasks \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "API Test Task",
    "type": "building",
    "priority": "normal"
  }'
```

You'll see the task appear immediately in the web UI!"

[NEXT STEPS - 9:00-10:00]

"Congratulations! You've successfully: - Installed HelixCode - Started the server - Created your first project and task - Used both the UI and API

In our next tutorial, we'll explore distributed worker pools and how to scale HelixCode across multiple machines. Thanks for watching!"

[RESOURCES] - Documentation: <https://docs.helixcode.dev> - GitHub: <https://github.com/helixcode/helixcode> - Community: <https://forum.helixcode.dev>

Tutorial 2: Aurora OS Deployment - Enhanced Security (15 minutes)

Script

[INTRO - 0:00-1:00]

"Welcome back! In this tutorial, we'll deploy HelixCode Aurora OS, our security-focused platform designed for Russian markets. Aurora OS provides enhanced security features including multi-level security configurations, comprehensive audit logging, and system monitoring."

[OVERVIEW - 1:00-2:00]

“Aurora OS offers three security levels: 1. Standard - Basic encryption and audit logging 2. Enhanced - Advanced access control, rate limiting, IP whitelisting 3. Maximum - Multi-factor auth, DLP, intrusion detection

Today we’ll deploy with Enhanced security, the recommended level for production.”

[DOCKER DEPLOYMENT - 2:00-5:00]

“The easiest way to deploy Aurora OS is with Docker Compose. First, clone the repository:

```
git clone https://github.com/helixcode/helixcode.git
cd HelixCode
```

Create your environment file:

```
cp .env.example .env
```

Edit the .env file with your secure passwords:

```
DATABASE_PASSWORD=your_secure_db_password
REDIS_PASSWORD=your_secure_redis_password
AURORA_SECURITY_LEVEL=enhanced
AURORA_AUDIT_LOGGING=true
```

Now start the services:

```
docker-compose -f docker-compose.aurora-os.yml up -d
```

Wait for the services to start. Check the status:

```
docker-compose -f docker-compose.aurora-os.yml ps
` `` ``
```

****[SECURITY CONFIGURATION - 5:00-8:00]****

"Let's verify our security configuration. Access the Aurora OS interface at <http://localhost:8080>.

Log in with your admin credentials. Navigate to Settings > Security. You'll see:

- Encryption: Active (AES-256-GCM)
- Audit Logging: Enabled
- Rate Limiting: 1000 req/hour
- Security Level: Enhanced

Let's enable MFA for additional security. Click 'Enable MFA' and follow the prompts to set up your authenticator app."

****[AUDIT LOGS - 8:00-10:00]****

"Aurora OS logs all security events. Navigate to Monitoring > Audit Logs. You can filter by:

- Event type (authentication, authorization, data access)
- Time range
- User
- Result (success/failure)

Let's view authentication events. Click 'Event Type' and select 'Authentication'. You'll see all login attempts with IP addresses, timestamps, and results. This is crucial for compliance and security monitoring."

****[SYSTEM MONITORING - 10:00-12:00]****

"Navigate to Monitoring > System to see real-time metrics:

- CPU usage and temperature
- Memory utilization
- Disk space
- Network traffic

You can set up alerts for threshold violations. Click 'Configure Alerts' and set:

- CPU > 80%: Email alert
- Memory > 90%: Slack notification
- Disk > 85%: Both

These alerts help you maintain system health."

****[PROMETHEUS INTEGRATION - 12:00-13:30]****

"Aurora OS exposes Prometheus metrics on port 9090. Let's verify:

```
```bash
curl http://localhost:9090/metrics
```

You'll see metrics in Prometheus format. To visualize these in Grafana, start the monitoring stack:

```
docker-compose -f docker-compose.aurora-os.yml --profile with-redis up -d
```

Access Grafana at <http://localhost:3000> (admin/admin). The Aurora OS dashboard is pre-configured!”

### **[BACKUP AND MAINTENANCE - 13:30-14:30]**

“Let’s create a backup:

# Backup database

```
docker exec helixcode-aurora-postgres pg_dump -U helixcode
helixcode > backup.sql
```

# Backup configuration

```
docker cp helixcode-aurora-os:/etc/helixcode/aurora-
config.yaml ./
```

# Backup logs

```
docker cp helixcode-aurora-os:/var/log/helixcode ./logs-backup
```

Schedule this with cron for regular backups.”

### **[WRAP UP - 14:30-15:00]**

“Excellent! You’ve deployed Aurora OS with enhanced security. You now have: - Encrypted production deployment - Comprehensive audit logging - Real-time system monitoring - Prometheus metrics integration - Automated backups

Next tutorial: Harmony OS distributed computing. See you there!”

---

## **Tutorial 3: Harmony OS - Distributed Computing (20 minutes)**

### **Script**

#### **[INTRO - 0:00-1:00]**

“Welcome to Harmony OS deployment! Harmony OS is HelixCode’s distributed computing platform designed for Chinese markets. It features AI acceleration, cross-device synchronization, and distributed task execution across multiple nodes.”

#### **[KEY FEATURES - 1:00-2:30]**

“Harmony OS provides: 1. Distributed Computing - Spread tasks across multiple workers 2. AI Acceleration - NPU/GPU support for AI workloads 3. Cross-Device

Sync - Real-time sync across devices 4. Service Discovery - Automatic worker detection 5. Auto-failover - Automatic recovery from failures

Perfect for large-scale AI development!”

### **[SINGLE NODE DEPLOYMENT - 2:30-5:00]**

“Let’s start with a single node:

```
cd HelixCode
cp .env.example .env
```

Configure Harmony OS in .env:

```
DATABASE_PASSWORD=secure_password
REDIS_PASSWORD=secure_password
HARMONY_DISTRIBUTED_ENABLED=true
HARMONY_AI_ACCEL=true
HARMONY_GPU_ENABLED=true
```

Deploy:

```
docker-compose -f docker-compose.harmony-os.yml up -d
```

Verify:

```
curl http://localhost:8080/health
```
```

****[DISTRIBUTED DEPLOYMENT - 5:00-9:00]****

“Now let’s add worker nodes. Start the full distributed stack:

```
```bash
docker-compose -f docker-compose.harmony-os.yml --profile
distributed up -d
```

This starts: - Master node (port 8080) - Worker 1 (port 8083) - Worker 2 (port 8084)

Check cluster status:

```
curl http://localhost:8080/api/v1/harmony/cluster/status
```

You’ll see all three nodes active!”

### **[AI ACCELERATION - 9:00-12:00]**

“Let’s configure AI acceleration. Navigate to Settings > AI Acceleration in the UI.

Enable: - GPU Acceleration (Mali-G78) - NPU Acceleration (Kirin 9000) - if available  
- Model Optimization (Quantization, Pruning) - Precision: FP16 for balanced performance

Click Save. Now check acceleration status:

```
curl http://localhost:8080/api/v1/harmony/ai/acceleration
```

You'll see NPU and GPU utilization metrics.”

### **[DISTRIBUTING TASKS - 12:00-15:00]**

“Create a distributed task via API:

```
TOKEN=$(curl -X POST http://localhost:8080/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"admin","password":"admin123"}' | jq -r
 '.token')
```

```
curl -X POST http://localhost:8080/api/v1/harmony/distribute \
-H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{
 "task_id": "task-123",
 "distribution_strategy": "round_robin",
 "required_capabilities": ["gpu", "ai_acceleration"]
}'
```

The task is distributed across workers with GPU capability!”

### **[CROSS-DEVICE SYNC - 15:00-17:00]**

“Enable cross-device sync. In the UI, go to Settings > Sync.

Configure: - Sync Interval: 30 seconds - Sync Types: Tasks, Sessions, Configurations  
- Devices: Add your phone and tablet

Click ‘Enable Sync’. Now changes sync automatically across all your Harmony OS devices!”

### **[MONITORING - 17:00-19:00]**

“View distributed metrics:

```
curl http://localhost:8080/api/v1/harmony/cluster/status
```

This shows: - Total nodes: 3 - Active tasks per node - Resource utilization - Task completion rates



Use Grafana for visualization:

```
docker-compose -f docker-compose.specialized-platforms.yml --
 profile monitoring up -d
```

Access at <http://localhost:3000!>”

**[WRAP UP - 19:00-20:00]**

“Amazing! You’ve deployed a full Harmony OS distributed cluster with: - 3-node distributed computing - AI acceleration (NPU/GPU) - Cross-device synchronization - Real-time monitoring

This setup can handle massive AI workloads. Thanks for watching!”

---

## Tutorial 4: Production Deployment Best Practices (25 minutes)

### Script

**[INTRO - 0:00-1:00]**

“In this advanced tutorial, we’ll deploy both Aurora OS and Harmony OS to production using the combined Docker Compose configuration, with full monitoring, high availability, and security hardening.”

**[TOPICS COVERED]** - Combined deployment architecture - SSL/TLS configuration - High availability setup - Monitoring with Prometheus and Grafana - Security hardening - Backup strategies - Performance optimization

**[CONTINUES WITH DETAILED PRODUCTION SETUP...]**

---

## Production Notes

### Video Requirements

- Resolution: 1920x1080 (1080p)
- Frame Rate: 30 FPS
- Format: MP4 (H.264)
- Audio: Clear narration, 128kbps AAC

### On-Screen Elements

- HelixCode logo (top right)

- Command line overlay for commands
- Highlight cursor and typing
- Use annotations for important points
- Include timestamps in video

## Post-Production

- Add chapter markers
- Include command list in description
- Link to documentation
- Provide code samples in GitHub

## Recording Tools Recommended

- OBS Studio (screen recording)
  - Audacity (audio cleanup)
  - DaVinci Resolve (editing)
- 

## Tutorial Release Schedule

1. **Week 1:** Getting Started (Tutorial 1)
  2. **Week 2:** Aurora OS Deployment (Tutorial 2)
  3. **Week 3:** Harmony OS Distributed (Tutorial 3)
  4. **Week 4:** Production Best Practices (Tutorial 4)
  5. **Week 5-8:** Advanced topics (CI/CD, Kubernetes, etc.)
- 

## Additional Tutorial Ideas

- **API Development:** Building applications with HelixCode API
- **Worker Pools:** Setting up SSH worker pools
- **LLM Integration:** Connecting to different LLM providers
- **Monitoring Setup:** Deep dive into Grafana dashboards
- **Security Hardening:** Advanced security configurations
- **Performance Tuning:** Optimizing for large-scale deployments
- **Kubernetes Deployment:** Running on K8s
- **CI/CD Integration:** GitHub Actions workflows
- **Troubleshooting:** Common issues and solutions
- **Migration Guide:** Upgrading from older versions